

CONTEXTGRAPH: MEASURING TRANSFER FROM EXPERIENCE TO VERIFIED CODE REPAIRS

Anonymous authors

Paper under double-blind review

ABSTRACT

Long-term memory can change what a coding agent reads without improving the patch it submits. We study this gap using ContextGraph, a system that turns prior repair evidence into rules and retrieves them through dense, lexical, and graph channels. Our evaluation separates memory provenance, actual delivery, agent execution, and official repair verification. Existing official reports for two completed SWE Context Bench Related-Lite conditions record 18 repairs each on the same 99 tasks, with six wins unique to each condition. The recorded paired difference is 0 percentage points (95% bootstrap interval $[-7.07, 7.07]$). A completed contemporaneous Control resolves 17/99; Flat and Card each gain only one observed repair over Control, with exact paired $p = 1$ for both comparisons. An extended audit identified one appended historical Docker log; an independent frozen-verifier recheck of the identical prediction reproduces the original failure, with no outcome changed. Actual memory is delivered on all tasks, while the correct procedure card reaches 98 of 99 targets. These observations do not establish a general limit on agent memory. We also prepare a separate SWE-bench Verified experiment that builds memory from 350 training tasks and compares no memory, flat rule retrieval, and graph rule retrieval on a fixed 150-task test set. This draft reports the audits, the retained provenance limitation, and the new experiment’s design; it does not yet report a test-set gain from the latter.

Draft status, 6 September 2026. This is a research draft, not a completed submission. Related-Lite first predictions have checked frozen-verifier evidence, including one independent recheck for the historical log exception described below. The separate Verified 350/150 inputs are frozen; execution stopped on an agent-framework error, and no formal effect estimate is available. Historical results from other repository experiments are not pooled into either denominator.

1 INTRODUCTION

Coding agents must locate relevant code, construct an explanation of a failure, edit a repository, and verify a repair. Prior experience could help at each stage, but a retrieval hit is not itself a successful repair. A memory system may return a correct example for the wrong software version, fail to deliver its output to the agent, or consume enough interaction budget to displace useful testing. Conversely, a low repair rate does not establish that the retrieved knowledge was unusable: the surrounding tool or evaluation protocol may have failed.

We investigate this distinction in ContextGraph, a long-term memory layer for coding agents. Its broader architecture combines typed experience nodes with lexical, embedding, and Personalized PageRank retrieval. The present study uses two distinct settings. The first is a retrospective audit of a completed repository-card intervention on SWE Context Bench Related-Lite (Zhu et al., 2026). The second is a new, deliberately narrower intervention on SWE-bench Verified (Jimenez et al., 2024): fixed rules derived from training issues and their official patches, with graph retrieval isolated from the rest of the retrieval stack. The latter is supervised memory construction with a frozen solver, not reinforcement learning or evidence of self-improvement from the solver’s own successful attempts.

Our current contributions are an evidence-linked evaluation procedure, an audited result for the repository-card intervention, and a controlled experiment specification for graph-based transfer. We do not claim that the audit alone establishes a new state of the art or that a null result identifies the

agent’s intrinsic capability ceiling. The scientific question is which forms of prior repair evidence improve verified outcomes after their delivery, cost, and source eligibility have been established.

2 RELATED WORK

Experience and graph retrieval. ExpeL extracts reusable natural-language insights from agent experience without requiring updates to proprietary model weights (Zhao et al., 2024). HippoRAG combines knowledge graphs and Personalized PageRank for associative retrieval (Gutiérrez et al., 2024). These works motivate ContextGraph’s representation and retrieval choices; they do not establish that the same mechanisms improve a coding agent’s repair success under a fixed interaction budget. SWE-agent demonstrates the importance of the agent–computer interface for software engineering (Yang et al., 2024), making interface fidelity part of our experimental specification.

Skills, working memory, and transfer. CODESKILL trains a skill-management mechanism while retaining a fixed coding policy; its reported Verified setup uses 350 training tasks and 150 evaluation tasks (Li et al., 2026). Our split size follows the same general design, but we do not claim to reproduce its task IDs, optimization procedure, or results. SWE-MeM studies adaptive working-memory management for long-horizon coding (Gao et al., 2026), whereas our new experiment supplies a fixed cross-task packet before solving. Agent Memory Distillation studies hierarchical teacher memory for smaller agents (Kim et al., 2026b); our experiment uses neither a small-model distillation objective nor teacher-weight training. Memory Transfer Learning distinguishes memory representations and reports that abstract knowledge can transfer more effectively than concrete traces in its evaluated settings (Kim et al., 2026a). This motivates a rule-based transfer hypothesis, without guaranteeing its outcome in our repositories.

Process measurements are intermediate outcomes. The Working Set studies coherence in repository-scale agent tasks (Mohammadi et al., 2026). Measure Before You Manage separates working-memory management and its evaluation (Chen et al., 2026a); its small policy-evaluation populations and incomplete official repair verification limit direct comparison with a fully graded SWE campaign. ClawTrace relates skill distillation to trajectory costs (Yuan et al., 2026). We use these works to motivate measurements of delivery, repeated errors, and consumption cost, while keeping official repair verdicts as the outcome of interest. We do not turn their setting-specific findings into universal claims about which memories help or harm.

Recent scaffold optimization also uses persistent beliefs: BCO calibrates a document against observed outcomes (Chen et al., 2026b). Its two memory-QA rows select the best held-out result among three candidates, a disclosed selection step that limits interpreting them as untouched test estimates. CANOPY instead trains the policy through outcome-only reinforcement learning (Pu et al., 2026); it is a distinct intervention from our frozen-model memory injection.

3 EVALUATION MODEL

For task i and condition a , let M_{ia} denote the supplied memory, D_{ia} actual delivery, T_{ia} the trajectory, and Y_{ia} the official verifier’s binary result. These objects are linked rather than inferred from one another:

$$\text{eligible sources} \longrightarrow M_{ia} \longrightarrow D_{ia} \longrightarrow T_{ia} \longrightarrow \text{submitted patch} \longrightarrow Y_{ia}.$$

Each link has a separate check. Source IDs must satisfy the frozen split; delivered content must appear in the actual model input or matching tool response; submitted patches and trajectories must match their recorded hashes; and a positive result must agree with both the task report and the harness summary. An infrastructure failure is a missing valid outcome, not an inferred negative result. An explicitly empty submission is a failure under the matching frozen verifier policy.

Fixed universes. An analysis uses its preregistered task list, never the intersection of whichever receipts happen to exist. If a campaign is incomplete, we report completion and missingness against that universe; we do not present a complete paired effect estimate. An infrastructure-only verifier repair is recorded separately and bound to the unchanged prediction. A valid failed attempt is not replaced by a later successful solver run.

Table 1: Audited Related-Lite first-prediction outcomes. The historical 88-task source-overlap sensitivity set does not replace the full 99-task denominator. Differences and exact two-sided McNemar tests are versus the contemporaneous Control on all 99 tasks. The historical Flat log exception remains disclosed in the text.

Condition	Fixed 99	Sensitivity 88	Difference	Exact p
Control	17/99	16/88	Reference	—
Flat	18/99	18/88	+1.01 pp	1.000
Card	18/99	15/88	+1.01 pp	1.000
Graph	12/99	12/88	−5.05 pp	0.227
Rules	13/99	12/88	−4.04 pp	0.424
File list	9/99	8/88	−8.08 pp	0.057

Delivery gates. Before interpreting a memory condition, at least 95% of targets must receive the expected memory and there must be no phase-based refusal. For the Related-Lite card condition, the expected card must match the target repository. Connection errors and an unbound string in a log do not count as delivery. Passing this gate establishes exposure, not relevance, correctness, or causal mediation through the memory.

Cost and protocol. Solver call counts, submitted completion status, and repair verdicts are different measurements. We record actual provider input/output tokens and charges where available. In local SWE-agent 1.1.0 diagnostics, retokenizing only assistant text omitted tool-call output from the reported output-token total, and local estimated costs differed from provider receipts. We therefore do not interpret unaudited legacy cost totals as measured savings. The local runtime also preserves the provider’s returned reasoning fields across tool turns and checks the options actually sent on the wire. These are protocol controls; the present observations do not establish a causal repair-rate effect of any individual protocol change.

4 COMPLETED RELATED-LITE AUDIT

Setting. The completed conditions use the same frozen 99-task Related-Lite manifest and the same frozen SWE Context Bench verifier. The flat condition retrieves reference patches from the existing source pool. The card condition adds a repository-specific procedure card: test commands, environment repair conditions, reproduction routines, and workflow information. Its sources include pool-derived evidence and environment recipes checked in benchmark containers. This is an intervention on repository environment knowledge, not an isolated test of graph traversal. The actual retrieved patch sets can vary with agent-generated queries, so identical serving logic does not imply byte-identical patch inputs across runs.

Receipt audit. All 99 receipts in each condition pass task-identity, literal-boolean verdict, trajectory/prediction hash, and report/summary consistency checks. For nonempty patches, the initial audit also checks the frozen harness evidence inventory, its file hashes, recorded offline Docker attestations, and runtime identity before and after verification. The harness commit is recorded in the accompanying evidence index. One task has a later verifier receipt, but its original receipt is already a valid failure; the analysis retains that original result.

An extended check of raw Docker logs flags Flat’s matplotlib-23687: its original 15 log records reproduce the receipt’s exact hash, but 21 later execution records have been appended. The original official report is unchanged and records failure (3/4 fail-to-pass and 791/791 pass-to-pass tests). The relationship of the appended execution records to that receipt is not fully established. A separate diagnostic verification of the identical prediction reproduces the original failure and passes the raw-log, frozen-runtime, report, and input-hash checks. It neither reruns the solver nor replaces the original outcome. We attach this independent evidence to that first prediction and retain the original provenance exception: the recheck does not reconstruct the historical execution. All reported first-prediction outcomes remain unchanged.

Outcomes. On the fixed universe, both conditions resolve 18/99 tasks (Table 1). Twelve successes are shared, six belong only to Flat, and six only to Card. The two-sided exact McNemar test gives $p = 1$, and the paired 95% bootstrap interval for Card minus Flat is $[-7.07, 7.07]$ percentage points. The interval is compatible with effects in both directions; lack of significance is not evidence of equivalence. These tasks have been used in multiple prior interventions, so this is an exploratory audit rather than a newly untouched confirmatory evaluation.

Source-overlap sensitivity. A pre-existing 11-task near-duplicate exclusion list yields an 88-task sensitivity set. Flat resolves 18/88 and Card 15/88; the discordant counts are six versus three, with $p = 0.508$ and interval $[-10.23, 3.41]$ percentage points. Three Card-only successes on the full set are flagged by the prior patch-overlap scan with Jaccard similarity 1. The sensitivity analysis applies the complete existing 11-task list, rather than removing only those three successful examples. The original scan focuses on resolved tasks; it does not certify all possible contamination channels over the entire task universe.

Actual exposure and mechanism checks. The tool-response census finds patch memory on 99/99 targets in each condition, no missing trajectories, and no phase refusals. The correct frozen repository card appears exactly in 98/99 Card trajectories. The exception is a scikit-learn task that receives a Matplotlib card; we retain its result in the fixed universe. The registered 95% exposure threshold is met, while this single routing failure remains part of the intervention’s observed behavior. All 198 debug logs are present, with no matches to the enumerated rate-limit error patterns; this narrow log check does not rule out every provider-side failure.

The hard `exit_cost` label occurs on 32 Flat tasks and 41 Card tasks, so Card does not meet its registered < 32 threshold. Both conditions have median 501 API calls under the legacy post-response call-limit behavior. The label can reflect the call cap, and must not be read as 41 dollar-budget overruns. An audit of all 198 first trajectories reproduces the archived test-output proxy: 81 Flat tasks and 73 Card tasks match, with conditional median first-match indices of 128 and 98 assistant messages. The proxy accepts failed tests, zero-test runs, and standalone OK prints; it is not a first-passing-test measure. Card meets the numerical timing threshold under this proxy but fails its coverage threshold. A conservative direct-runner and positive-execution-summary census recognizes 78/99 Flat tasks and 67/99 Card tasks, with conditional median indices of 116.5 and 97 recorded trajectory entries. Among the 60 tasks recognized in both conditions, Card is earlier on 38 and later on 22. This census omits some real tests in unsupported command forms; its step counter also differs from the archived counter, and counted unittest loader placeholders require the sensitivity check below. Conditional timing and incomplete recognition do not establish an overall speedup, test relevance, or a cause of the unchanged repair score.

The available gold-file call recognizer matches 61/99 Flat tasks and 49/99 Card tasks, with conditional median indices 212 and 165 assistant messages. Card meets this numerical median threshold on a smaller recognized subset. An exact-path-spelling sensitivity leaves Flat unchanged and recognizes 48 Card tasks, with median 161; the removed Card call names a different `__init__.py` and explicitly fails to replace its text. Both measures recognize mutation-like commands, including failed calls and incidental mentions, rather than certifying successful source edits.

Protocol diagnostic. Two literal thinking/tool markers occur inside submitted actions on 12/99 Flat tasks and 19/99 Card tasks; their paired observations contain syntax-error or cancellation messages. On one fixed completed task in each of Flat, Card, and Control, 13/15 sampled logged responses contain independent reasoning content, while the stored assistant histories omit those fields. The inspected client also omits them when constructing messages, although this source check is not a capture of transmitted requests. These findings document execution limitations; they do not establish why the markers appeared, whether reasoning loss caused them, or their effect on repair performance.

Other archived interventions. The extended audit also covers a graph-channel variant, an added-rule variant, and an added-file-list variant. All 99 prediction/verifier chains in each of these three conditions pass its checks, with 12, 13, and 9 recorded first-attempt repairs, respectively; eleven later solver reruns in the rule condition are not substituted. Their actual patch-memory exposure is 99/99, 95/99, and 96/99, with no phase refusals; the latter two meet their registered requirements for rule

and file-list presence. Eleven SymPy tasks in the file-list condition receive paths with Matplotlib, Sphinx, or Astropy prefixes, illustrating that delivered candidates need not be useful repository locations. The rule retriever and file-list design used offline metrics on these same 99 targets during method selection. Together with runtime failures and the retained Flat provenance limitation, this prevents treating the archived counts as a clean confirmatory comparison of memory mechanisms. In a retrospective family of four comparisons against Flat, Holm-adjusted exact McNemar p values are 1.000 (Card), 0.094 (Graph), 0.453 (Rules), and 0.047 (file list). The file-list difference is -9.09 pp with a paired bootstrap interval $[-16.16, -3.03]$ percentage points; this interval is not adjusted for multiple comparisons. The observed difference does not isolate which aspect of the intervention or execution caused it. Figure 1 shows the paired estimates for both fixed task universes.

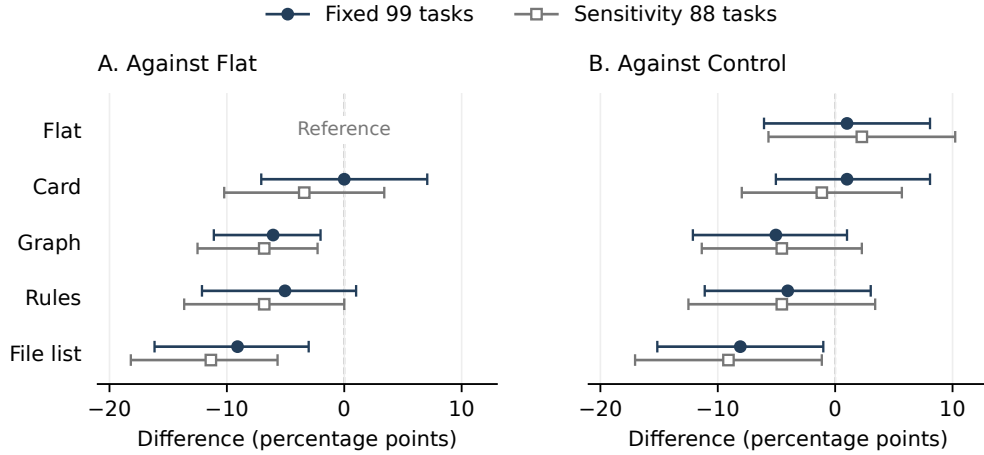


Figure 1: Archived Related-Lite first-prediction differences relative to Flat (left) and the contemporaneous Control (right). Bars are unadjusted 95% paired percentile-bootstrap intervals (200,000 draws; seed 42), not inversions of the exact McNemar tests. These retrospective comparisons include operational failures and retain the historical receipt limitation; they do not isolate causal memory effects.

Control comparison. The contemporaneous Control now has 99 checked first-prediction receipts and resolves 17/99. Its exact corresponding trajectories show no enumerated memory markers in initial prompts and no explicit memory calls; this is a direct-exposure check, not a general contamination certificate. Flat and Card each exceed Control by one observed repair ($+1.01$ pp), with respective paired bootstrap intervals $[-6.06, 8.08]$ and $[-5.05, 8.08]$ percentage points and exact $p = 1$ for both. Neither comparison shows a stable gain, and neither establishes equivalence. In the retrospective five-comparison Holm family against Control, adjusted p values are 1.000 (Flat), 1.000 (Card), 0.906 (Graph), 1.000 (Rules), and 0.287 (file list). The file-list percentile interval excludes zero but does not override its exact test ($p = 0.057$) or that correction. The complete historical sensitivity list leaves Control at 16/88, Flat at 18/88, and Card at 15/88. Control has 48 hard `exit_cost` labels, versus 32 for Flat and 41 for Card; these labels can denote the call cap. We retain runtime failures and empty submissions, never substitute later solver runs, and do not reuse the historical 14/99 Control.

The same original test-summary recognizer covers 61/99 Control tasks, with conditional median first-match index 134. A deterministic source check finds that `unittest` can report a positive test count for an artificial `_FailedTest` when the named test cannot be loaded. A uniform 297-trajectory sensitivity excludes observations with enumerated loader-placeholder markers: recognized coverage is then 61/99 Control, 78/99 Flat, and 66/99 Card, with conditional medians 146, 116.5, and 97.5. This preserves the original statistics and all repair outcomes; the revised summaries still do not certify test-body execution or overall speedup.

5 VERIFIED 350/150 EXPERIMENT

Split and eligibility. This campaign is separate from Related-Lite. We pin the official 500-task Verified dataset and deterministically allocate 350 training and 150 test tasks using repository-stratified largest-remainder allocation and a fixed hashed ordering (seed 42). The two ID sets are disjoint. The split is within repositories and is not chronological; it does not establish that test tasks are absent from the base model’s training history or earlier project experiments. One issue-text near-duplicate pair is documented before formal solving; excluding its test member defines a fixed 149-task sensitivity analysis. The primary universe remains all 150 tasks.

Memory construction. Only training issues and their official gold patches are used to derive rules. Each extracted record carries source identity and evidence references, with mechanical coverage and evidence-line checks. The complete bank contains 459 rules from 350 sources and 61 mechanism tags. Passing these checks does not prove that every generalized rule is semantically valid; semantic overgeneralization remains a limitation. A deterministic training-only sample of 30 rules received a source-reading AI audit: 20 were judged supported and 10 partly supported, with core edits grounded but errors in explanation or scope. Examples include widening an exact Boolean predicate to truthiness and reversing the explanation of which join comparison includes a filtered condition. These advisory judgments are neither blinded human annotations nor a correctness estimate for the full bank; the frozen bank and evaluation packets remain unchanged. Embeddings are computed locally with a pinned 384-dimensional MiniLM model, without additional embedding API calls. The solver weights remain fixed.

Three conditions. Control receives no reference memory. Flat and Graph share the same rule corpus, dense search, lexical search, rank fusion, diversity reranking, top-three limit, and text budget. Graph additionally enables the Personalized PageRank channel over the fixed rule/source/mechanism structure. All memory packets are constructed from public test descriptions before formal solver calls, then frozen; there are no test-time memory writes. Each packet is inserted once into the initial task input. This narrows the estimand to the effect of enabling the graph channel in this implementation, not the effect of every component in the broader ContextGraph system.

The frozen packet contrast confirms a real intervention: the ordered rule IDs differ on 94/150 tasks, and the top-three sets differ on 87/150. All 150 tasks remain in the primary comparison, including those receiving identical packets. Restricting the reported result to tasks whose retrieval changes would answer a different question and is not the primary analysis.

Execution controls. All three conditions use SWE-agent 1.1.0 and the exact DeepSeek V4 Pro endpoint, with no alternate-model fallback. The frozen per-attempt limits are 500 completed calls and a \$30 conservative budget-meter threshold, with equal generation settings and preserved reasoning history. A \$35 per-attempt reservation accounts for the post-response check; the OpenLux route has no provider-enforced price filter. The campaign’s \$300 admission allowance includes \$20 for monitoring and may leave the campaign incomplete. Task blocks contain all three conditions; their six possible orders occur 25 times each across the 150 tasks. The first block uses one worker to measure resources, after which up to four may run if the host and campaign budget permit. No task is admitted without a reservation for its complete three-condition block.

Solver containers receive only public task fields, the frozen memory packet, and a sealed base-commit checkout. The Git seal preserves the base tree and its ancestors while removing future or unreachable commit objects. The solver’s Docker network has no external egress. Credentials remain on the host, and evaluation labels are consumed only by the host-side verifier. Each submission is immediately evaluated with the pinned official SWE-bench 4.1.0 harness and its task-specific original image. Verifier containers have the same two-CPU/eight-GiB cap across all conditions; grading logic is unchanged. Runtime, configuration, tool bundle, image, and source hashes are checked before and after execution. Ten-minute reviews inspect trajectories and execution progress without editing the frozen treatment.

Analysis plan. The primary contrast is Graph minus Flat, and the secondary contrast is Flat minus Control. We use two-sided exact McNemar tests with Holm correction over these two registered comparisons, and paired-task bootstrap confidence intervals with 200,000 replicates and seed 42.

We report outcomes against the full 150-task universe, plus the predeclared 149-task sensitivity set. Raw provider usage and a separate conservative budget meter are retained; unreconciled estimates do not establish cost efficiency. Other repository or task slices are exploratory. Budget or infrastructure interruption preserves all artifacts and leaves the campaign incomplete; it does not shrink the primary denominator.

Current evidence. The 350-source bank and 450 memory packets are complete. Training-only real-agent runs exercise memory delivery, the provider protocol, and the official verifier before formal deployment. Their task is excluded from the smoke memory bank and their outcomes are not test-set observations. All 150 images and 450 task/condition inputs are frozen; the original provider passed all three training-only gates before an account-credit error paused launch. A subsequently authorized OpenLux route uses the same DeepSeek V4 Pro snapshot alias as the archived experiments, preserving the split, packets, images, and task order in a separate runtime version. Its tool/reasoning probes and all three training protocol/delivery checks passed. On the excluded training task, Control’s original prediction was officially resolved; Flat’s empty patch and Graph’s reproduction-only patch were unresolved. The formal campaign began on September 6 at 11:21 UTC. The original scheduler closed at 12:52 UTC with two complete blocks: all three conditions resolve Django11740 and none resolves Xarray6992. A seventh attempt, Django16256 Flat, auto-submits an empty patch after consecutive command timeouts and is officially unresolved; its outer status classification stops new-block admission. The original registered analysis remains incomplete and has no paired effect estimate. A separately recorded amendment then resumes only previously unstarted slots, accepting ordinary agent-stop outcomes when the original protocol, delivery, prediction and verifier audits pass. It retains all seven original attempts and terminal states, the 150-task denominator, and the original runtime, inputs, limits and budget policy. This post-start interpretation change is disclosed separately; any eventual combined analysis is not an untouched confirmatory experiment. The amendment closed at 14:05 UTC after four complete blocks when a Sphinx Control response supplied a null optional array argument that crashed the agent’s tool parser. Its 47 provider calls pass the protocol audit, but the generic error stop remains inadmissible and the official receipt records an empty patch. Fourteen original slots were attempted, thirteen pass the full audit, and 436 remain unstarted; no failed attempt is replaced. A separately frozen parser correction passes all three training protocol/delivery gates: Control and Flat resolve the excluded training task, while Graph submits an empty patch. All three runs record zero optional-null omissions; the recorded-call regression supplies coverage of the original defect. The formal test runtime and its original outcomes remain unchanged. OpenLux sometimes omits cost and reports non-additive token counts, so raw counters and conservative budget estimates are stored separately without a billing claim. Ten-minute evidence collection continues, with incomplete semantic reviews explicitly retained. No formal Verified success rate is reported in this draft.

6 DISCUSSION AND LIMITATIONS

The completed audit rules out a particular ambiguity: the card intervention’s lack of measured gain cannot be explained by widespread absence of any memory response, because actual delivery is observed. It does not rule out incorrect relevance judgments, stale environment knowledge, variable retrieval queries, different provider behavior, or insufficient repair skill. Exposure is a prerequisite for evaluating an intervention, not a proof of its proposed mechanism.

A training-only case illustrates conditional transfer. Flat and Graph both retrieve two rules from Requests1724, whose Python 2 Unicode-method repair adds `builtin_str(method)`. The excluded training task Requests2317 instead reports a Python 3 bytes-method failure at that line; its official training patch replaces the call with `to_native_string(method)`. The retrieved rules already state the Python 2 condition. In the new Graph training run, the agent observes the correct conversion output but ultimately submits an empty patch at its budget limit. Flat, which receives both source rules as well, is officially resolved on the same training task. This documents version dependence and a gap between observing a relevant operation and submitting a repair; it does not establish that memory caused the failure or that an applicability filter improves outcomes. A narrow, case-motivated scan of all 350 training patches recovers this pair only; this is neither an independently sampled case nor an estimate of prevalence.

The new Verified design reduces several confounds by sharing the rule corpus, packet size limits, model settings, and task order policy while varying one retrieval channel. It still has important limits: a single model endpoint, one stochastic attempt per condition, a within-repository random split, official training-patch supervision, and potential historical task exposure. The new protocol also changes multiple execution details relative to historical experiments. Consequently, a future difference from an old score would not isolate the effect of memory.

A stronger causal account would require an independent environment-by-memory intervention, repeated attempts or additional model backbones, and process measures that are validated against real test execution. Such work should use training/development evidence for method selection and freeze evaluation eligibility before inspecting outcomes. The aspiration to improve memory should not make success the inclusion criterion for evidence.

REPRODUCIBILITY STATEMENT

Frozen task lists, source and runtime hashes, per-task official receipts, and the evidence map support the reported claims. Two local anonymous supplements reproduce the 18 Related-Lite paired comparisons and 594 recorded-exposure checks using source-bound verdicts, selected history fields and the original analysis code. The latter retains actual memory responses, expected frozen cards and the same trajectory hashes used for verdicts. These packages do not rerun official verification or debug-log rate-limit checks. Credentials and host-only evaluation labels are excluded from agent inputs. The complete anonymous experimental release and human author review remain to be completed.

AI USE STATEMENT

Generative AI tools were used to assist literature discovery and interpretation, formulate and refine hypotheses, design experiments, implement and test code, extract training-memory rules, audit trajectories, analyze results, and draft this manuscript. DeepSeek V4 Pro is also the experimental solver and the source-rule extraction model in the new Verified campaign. Automated progress reviews are advisory; official harness receipts determine repair outcomes. This draft has not yet completed human author review of all AI-assisted claims and artifacts. The final disclosure must reflect the review actually performed before submission.

REFERENCES

- Le Chen, Zishen Wan, Baixi Sun, Xiaolong Ma, Chih-Hsuan Yang, Feng Yan, Sheng Di, Franck Cappello, and Rajeev Thakur. Measure before you manage: Evaluating agent working memory in coding agents, 2026a. URL <https://arxiv.org/abs/2608.31057v1>. Version 1.
- Yuhan Chen, Zhihua Tian, Mahavir Dabas, Charith Peris, Rahul Gupta, Ming Jin, Feiyang Kang, Siyuan Zhang, Nan Wang, and Ruoxi Jia. Belief-calibrated optimization: An explicit world model for agentic optimization, 2026b. URL <https://arxiv.org/abs/2609.01861v1>. Version 1.
- Shuzheng Gao, Wenhao Zeng, Zhaojian Yu, Jianqiao Wangni, Chaozheng Wang, Kai Cai, Shilin He, and Michael R. Lyu. Swe-mem: Learning adaptive memory management for long-horizon coding agents, 2026. URL <https://arxiv.org/abs/2606.28434v1>. Version 1.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models, 2024. URL <https://arxiv.org/abs/2405.14831v3>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Kangsan Kim, Minki Kang, Taeil Kim, Yanlai Yang, Mengye Ren, and Sung Ju Hwang. Memory transfer learning: How memories are transferred across domains in coding agents, 2026a. URL <https://arxiv.org/abs/2604.14004v1>. Version 1.

- Taeil Kim, Kangsan Kim, and Sung Ju Hwang. Agent memory distillation: Empowering small llm agents with hierarchical teacher memory, 2026b. URL <https://arxiv.org/abs/2608.07169v1>. Version 1.
- Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. Codeskill: Learning self-evolving skills for coding agents, 2026. URL <https://arxiv.org/abs/2605.25430v1>. Version 1.
- Bardia Mohammadi, Lars Klein, Aman Chadha, Akhil Arora, and Laurent Bindschaedler. The working set of a coding agent: Coherence debt in repository-scale tasks, 2026. URL <https://arxiv.org/abs/2608.16630v1>. Version 1.
- Liming Pu, Xiaoxia Li, Yifu Liu, Teng Cao, and Bin Yang. Explore more, drift less: Outcome-only reinforcement learning can suffice for long-horizon interactive agents, 2026. URL <https://arxiv.org/abs/2609.01245v1>. Version 1.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Boqin Yuan, Yue Su, Renchu Song, Sen Yang, and Jing Qin. Clawtrace: Cost-aware tracing for llm agent skill distillation, 2026. URL <https://arxiv.org/abs/2604.23853v2>. Version 2.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.
- Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. Swe context bench: A benchmark for context learning in coding, 2026. URL <https://arxiv.org/abs/2602.08316v3>. Version 3.